

NAME: CH. Durga Prasad	REG NO: 192425305
COURSE: MLA0407 DEEP LEARNING	FACULTY: Dr. AMANDEEP SINGH

Experiment 1:

Aim: To demonstrate confusion matrix using python

```

import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

# Actual and Predicted labels
actual = np.array([
    'Dog', 'Dog', 'Dog', 'Not Dog', 'Dog',
    'Not Dog', 'Dog', 'Dog', 'Not Dog', 'Not Dog'
])

predicted = np.array([
    'Dog', 'Not Dog', 'Dog', 'Not Dog', 'Dog',
    'Dog', 'Dog', 'Dog', 'Not Dog', 'Not Dog'
])

# Create Confusion Matrix
conf_matrix = confusion_matrix(actual, predicted)

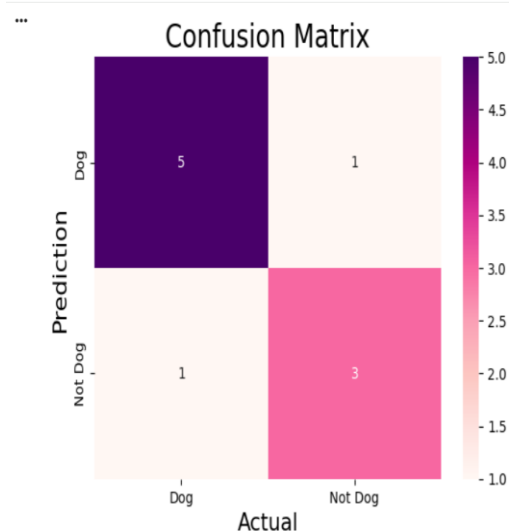
# Plot Confusion Matrix
sns.heatmap(
    conf_matrix,
    annot=True,
    fmt='g',
    cmap='RdPu',
    xticklabels=['Dog', 'Not Dog'],
    yticklabels=['Dog', 'Not Dog']
)

plt.xlabel("Actual", fontsize=16)

plt.xlabel("Actual", fontsize=16)
plt.ylabel("Prediction", fontsize=16)
plt.title("Confusion Matrix", fontsize=20)

plt.show()

```



Experiment 2:

Aim: To demonstrate 2 class confusion matrix using python

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

# Load dataset
wine = load_wine()

# Create DataFrame
data = pd.DataFrame(data=wine.data, columns=wine.feature_names)
data["Target"] = wine.target

# Keep only two classes (0 and 1)
data = data[data["Target"] != 2]

# Features and Target
X = data.drop("Target", axis=1)
y = data["Target"]

# Split dataset
```

Accuracy: 0.9743589743589743

Classification Report:

```
...      precision    recall  f1-score   support

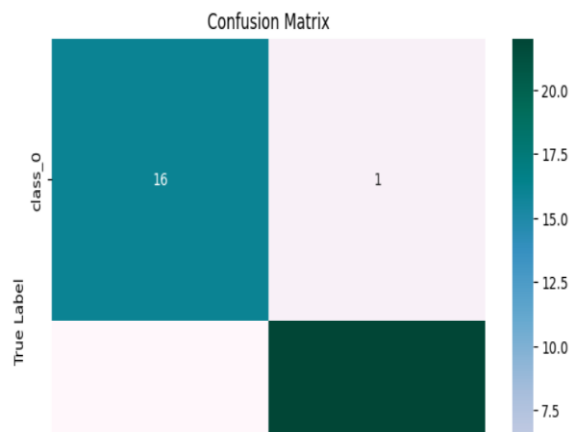
...   class_0      1.00      0.94      0.97        17
...   class_1      0.96      1.00      0.98        22

...   accuracy              0.97        39
...  macro avg       0.98      0.97      0.97        39
... weighted avg       0.98      0.97      0.97        39
```

Precision: 0.96

Recall: 1.00

F1 Score: 0.98



Experiment 3:

Aim: To analyse the performance of a multi class confusion matrix by using chosen database with python code

```
0 from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

import seaborn as sns
import matplotlib.pyplot as plt

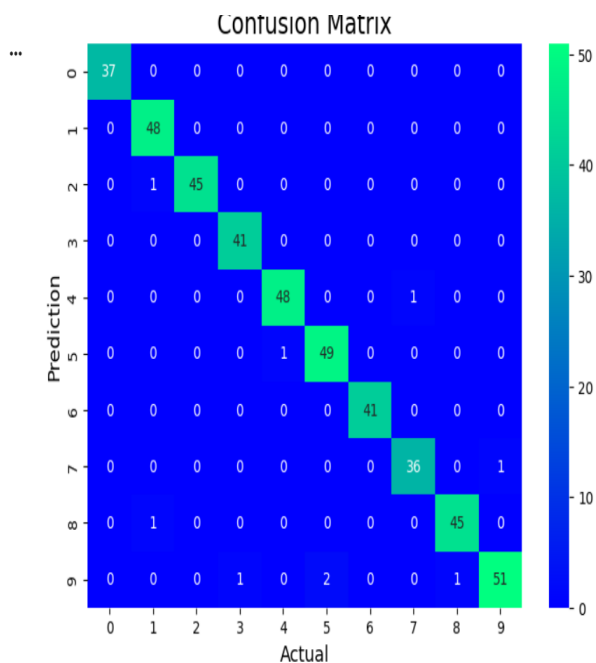
# Load the Digits dataset
X, y = load_digits(return_X_y=True)

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=23
)

# Create and train the model
clf = RandomForestClassifier(random_state=23)
clf.fit(X_train, y_train)

# Make predictions
y_pred = clf.predict(X_test)

# Create confusion matrix
cm = confusion_matrix(y_test, y_pred)
```



Accuracy : 0.98
Precision: 0.9801896286719816
Recall : 0.98

Experiment 4:

Aim: To analyse the performance of a over fitting by using choosen database with python code

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score

# True function
def true_fun(X):
    return np.cos(1.5 * np.pi * X)

# Generate data
np.random.seed(0)

n_samples = 30
degrees = [1, 4, 15]

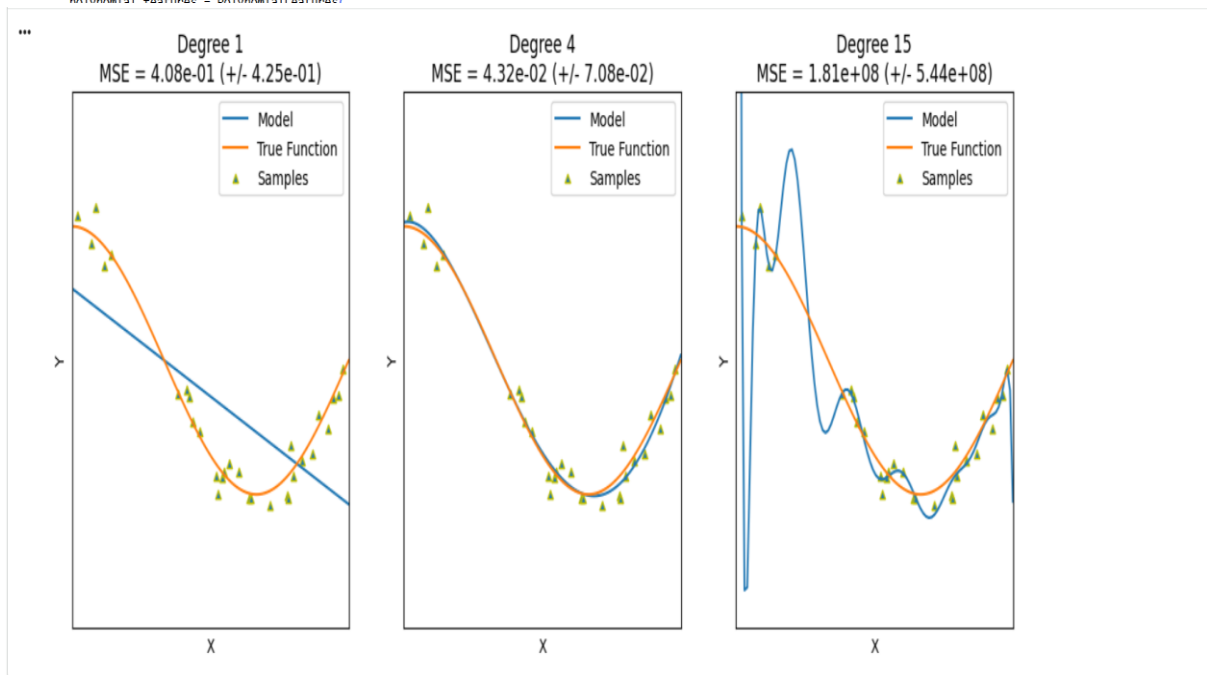
X = np.sort(np.random.rand(n_samples))
y = true_fun(X) + np.random.randn(n_samples) * 0.1

# Plot results
plt.figure(figsize=(14, 5))

for i in range(len(degrees)):
    ax = plt.subplot(1, len(degrees), i + 1)
    plt.setp(ax, xticks=(), yticks=())

    # Create polynomial regression model
    polynomial_features = PolynomialFeatures(degree=degrees[i])
    model = LinearRegression()
    pipeline = Pipeline([
        (polynomial_features, model)
    ])
    pipeline.fit(X, y)

    # Print MSE
    mse = cross_val_score(pipeline, X, y, cv=5, scoring='neg_mean_squared_error')
    mse = -mse
    print(f'Degree {degrees[i]} MSE = {mse[0]:.2e} (+/- {mse[1]:.2e})')
```



Experiment 5:

Aim: To demonstrate the performance of a linear regression by using chosen database with python code

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load Iris dataset
iris = load_iris()

# Create DataFrame
data = pd.DataFrame(data=iris.data, columns=iris.feature_names)
data["species"] = iris.target

# Display first 5 rows
print(data.head())

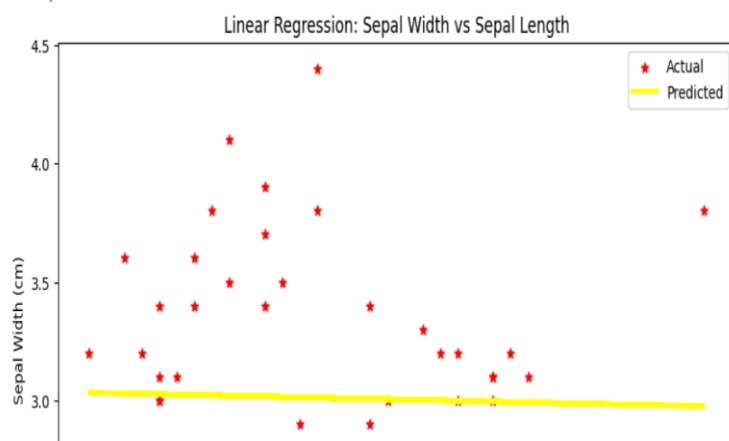
# Select features and target
X = data[["sepal length (cm)"]]
y = data["sepal width (cm)"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Train Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
...
sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1                3.5              1.4              0.2
...
1                4.9                3.0              1.4              0.2
2                4.7                3.2              1.3              0.2
3                4.6                3.1              1.5              0.2
4                5.0                3.6              1.4              0.2
```

```
species
0      0
1      0
2      0
3      0
4      0
Mean Squared Error on test set: 0.23
```



Experiment 6:

Aim: To demonstrate the performance of knn using wine dataset

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Load Wine dataset
wine = load_wine()

# Create DataFrame
data = pd.DataFrame(data=wine.data, columns=wine.feature_names)
data["Target"] = wine.target

# Features and Target
X = data.drop("Target", axis=1)
y = data["Target"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)

# Train KNN model
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
```

... Accuracy: 0.7037037037037

